

Build a Complete Sales and Inquiry Forms Module for the Event Platform

We need to design and develop a new module in our multi-tenant event management platform called **Sales**.

The platform is an existing Laravel-based application. Before writing code, inspect the codebase carefully and identify:

- Laravel version and current architectural patterns
- Frontend technology and component structure
- Existing database conventions
- Multi-tenancy implementation
- Organization and event-level scoping
- Authentication and authorization patterns
- Existing custom question and conditional question functionality
- Existing table, detail page, form builder, modal, drawer, and Kanban components
- Existing APIs, services, repositories, actions, jobs, events, notifications, and policies
- Existing conventions for migrations, models, controllers, requests, resources, enums, permissions, routes, and tests

Do not introduce a completely new architectural style unless necessary. Reuse existing patterns and components wherever possible.

The Sales module will contain two major sections:

1. Sales Pipelines and Deals
2. Website Inquiry Forms and Form Submissions

The implementation must be modular, scalable, secure, event-scoped, and reusable.

Part 1: Sales Pipelines and Deals

1. Sales Module Navigation

Add a new main navigation section called:

Sales

The Sales module should contain the following sub-sections:

- Dashboard
- Pipeline Types

- Pipelines
- Deals
- Inquiry Forms
- Form Submissions, if a separate global submissions screen is useful

Access must be controlled through permissions.

Suggested permissions:

- sales.view
- sales.dashboard.view
- sales.pipeline-types.view
- sales.pipeline-types.create
- sales.pipeline-types.update
- sales.pipeline-types.delete
- sales.pipelines.view
- sales.pipelines.create
- sales.pipelines.update
- sales.pipelines.delete
- sales.deals.view
- sales.deals.create
- sales.deals.update
- sales.deals.delete
- sales.deals.move-stage
- sales.deals.assign
- sales.deals.export
- sales.forms.view
- sales.forms.create
- sales.forms.update
- sales.forms.delete
- sales.forms.publish
- sales.form-submissions.view
- sales.form-submissions.update
- sales.form-submissions.delete
- sales.form-submissions.convert-to-deal
- sales.form-submissions.export

Use the platform's existing permission naming and authorization conventions if they differ.

2. Pipeline Types

A Pipeline Type works as a reusable template or configuration for pipelines.

Examples:

- Sponsorship Sales

- Exhibitor Sales
- Group Registration Sales
- Partnership Sales
- Vendor Sales
- General Event Sales

A Pipeline Type must define:

- Name
- Internal code or slug
- Description
- Status: active or inactive
- Color or visual identifier, if supported
- Pipeline stages
- Pipeline-level custom fields
- Deal-level custom fields
- Default stage
- Won stage or stages
- Lost stage or stages
- Stage ordering
- Optional default deal probability per stage
- Optional pipeline creation questions
- Optional form mapping configuration
- Created by
- Updated by
- Organization scope
- Event scope, depending on the current tenancy model

Pipeline Types should not directly contain deals. They define the structure that individual pipelines and deals will follow.

Pipeline Type Listing Screen

Create a listing page with:

- Search
- Status filter
- Created date filter
- Sort
- Pagination
- Create button
- Edit action
- Duplicate action
- Activate or deactivate action
- Delete action
- View detail action

Display:

- Name
- Number of stages
- Number of pipeline custom fields
- Number of deal custom fields
- Number of pipelines created from this type
- Status
- Updated date

Prevent accidental deletion when pipelines already use the Pipeline Type.

Preferred behavior:

- If the Pipeline Type is unused, allow deletion.
- If it is already used, allow deactivation or archive it instead.
- Do not break existing pipeline and deal data.

3. Pipeline Type Stages

Each Pipeline Type must have configurable stages.

For every stage, support:

- Name
- Internal code or slug
- Description
- Color
- Display order
- Probability percentage
- Stage category:
- Open
- Won
- Lost
- Is default stage
- Is active
- Optional expected response time or SLA
- Optional stage instructions
- Optional required fields before moving into this stage

Examples for Sponsorship Sales:

1. New Lead
2. Contacted
3. Qualified

4. Proposal Sent
5. Negotiation
6. Contract Sent
7. Won
8. Lost

Examples for Exhibitor Sales:

1. Inquiry Received
2. Qualification
3. Booth Options Shared
4. Quotation Sent
5. Follow-up
6. Confirmed
7. Lost

The user should be able to:

- Add a stage
- Edit a stage
- Delete an unused stage
- Reorder stages using drag and drop
- Mark a stage as default
- Mark stages as Won or Lost
- Configure stage color
- Configure stage probability
- Configure required deal fields for a stage

Validation rules:

- At least one stage must exist.
- Exactly one default stage should exist.
- Stage order must remain unique.
- A stage used by existing deals should not be hard-deleted without safe migration.
- Before deleting a used stage, require the administrator to select another stage to which existing deals will be moved.
- At least one Won stage and one Lost stage are recommended, but determine whether they should be mandatory based on existing product conventions.

4. Custom Fields for Pipeline Types

Pipeline Types can define two types of custom fields:

A. Pipeline-Level Fields

These fields are asked when creating a pipeline from a Pipeline Type.

Example:

For a Sponsorship Pipeline Type:

- Event edition
- Sales target
- Currency
- Sponsorship brochure URL
- Primary sales manager
- Sales start date
- Sales deadline

For an Exhibitor Pipeline Type:

- Hall
- Booth inventory category
- Pricing plan
- Target exhibitor industry
- Sales region

B. Deal-Level Fields

These fields appear when creating or editing deals under pipelines belonging to the Pipeline Type.

Examples:

- Company name
- Package type
- Requested booth size
- Sponsorship category
- Estimated amount
- Industry
- Country
- Lead source
- Budget range
- Expected closing date
- Company website
- Number of attendees
- Preferred package
- Additional requirements

Use or extend the platform's existing custom question and conditional question system.

Supported field types should include, based on current platform capabilities:

- Short text
- Long text

- Email
- Phone
- Number
- Currency
- Percentage
- Date
- Date and time
- Dropdown
- Multi-select
- Radio buttons
- Checkboxes
- Yes or No
- URL
- File upload
- User selector
- Contact selector
- Country selector
- Industry selector
- Hidden field
- Static information block, if already supported

Each field should support:

- Label
- Internal key
- Description or help text
- Placeholder
- Default value
- Required or optional
- Active or inactive
- Sort order
- Validation rules
- Minimum and maximum values
- Minimum and maximum length
- Allowed file types and file size
- Options for choice fields
- Conditional visibility
- Conditional requirement
- Whether shown on create form
- Whether shown on edit form
- Whether shown in deal detail
- Whether shown in the listing table
- Whether searchable
- Whether filterable
- Whether exportable
- Whether required at a particular stage

Conditional rules should support operators such as:

- Equals
- Does not equal
- Contains
- Does not contain
- Is empty
- Is not empty
- Greater than
- Less than
- Includes any
- Includes all

Example:

- Show “Booth Size” only when “Inquiry Type” equals “Exhibitor”.
 - Require “Proposal Amount” when the deal moves to “Proposal Sent”.
 - Require “Lost Reason” when the deal moves to a Lost stage.
-

5. Pipelines

A Pipeline is an actual sales process created from a Pipeline Type.

Examples:

- Web Summit 2026 Sponsorship Pipeline
- Tech Trip 2.0 Exhibitor Pipeline
- Annual Conference Group Sales Pipeline

Pipeline Creation Flow

When creating a pipeline:

1. Select the Pipeline Type.
2. Once selected, dynamically display the Pipeline-Level Fields configured for that Pipeline Type.
3. Enter the pipeline’s general information.
4. Assign responsible users or teams.
5. Save the pipeline.

Standard pipeline fields:

- Pipeline name
- Pipeline Type
- Description
- Status

- Start date
- End date
- Currency
- Revenue target
- Owner
- Assigned team members
- Visibility
- Pipeline-level custom field answers
- Created by
- Updated by

Once created, the pipeline should inherit:

- Pipeline Type stages
- Stage order
- Stage colors
- Stage categories
- Stage probabilities
- Deal custom fields
- Stage-level required fields

Important architectural decision:

Determine whether stage definitions should be referenced directly from Pipeline Type stages or copied into pipeline-specific stages.

Recommended behavior:

- Create pipeline-specific stage records from the Pipeline Type stage templates when a pipeline is created.
- This allows a pipeline to retain historical consistency even when the Pipeline Type is updated later.
- Pipeline Type changes should not silently alter active pipelines.
- Optionally provide an explicit “Sync changes from Pipeline Type” action later.
- Document the final decision in the implementation plan.

Pipeline Listing

Display:

- Pipeline name
- Pipeline Type
- Owner
- Number of deals
- Open deal value
- Won deal value
- Lost deal value
- Revenue target

- Progress percentage
- Status
- Start and end dates
- Last activity date

Filters:

- Pipeline Type
- Owner
- Status
- Date range
- Event
- Organization
- Has open deals
- Has overdue deals

Actions:

- View
- Edit
- Duplicate
- Archive
- Delete if empty
- View Kanban
- Create deal

6. Pipeline Detail Screen

Create a comprehensive pipeline detail screen.

Suggested tabs:

- Overview
- Deals
- Kanban
- Activities
- Team
- Settings

Overview

Show:

- Pipeline name
- Pipeline Type

- Description
- Owner
- Team
- Status
- Date range
- Currency
- Revenue target
- Total deal value
- Weighted pipeline value
- Won value
- Lost value
- Open value
- Number of deals
- Conversion rate
- Average deal value
- Average sales cycle
- Custom field values

Deals Tab

Show all deals belonging to the pipeline in a table.

Kanban Tab

Show deals grouped by pipeline stage.

Activities Tab

Show recent activity such as:

- Deal created
- Deal updated
- Deal moved
- Contact added
- Note added
- Owner changed
- Deal marked won
- Deal marked lost
- Form submission converted to deal

Team Tab

Manage pipeline owner and assigned users.

Settings Tab

Allow authorized users to edit pipeline details and possibly manage pipeline-specific stage settings.

7. Deals

A Deal belongs to one pipeline.

Because each pipeline belongs to one Pipeline Type, the Deal form and available stages are determined by that Pipeline Type and the pipeline's copied stage configuration.

Standard Deal fields:

- Deal title
- Deal number or reference
- Pipeline
- Current stage
- Deal value
- Currency
- Probability
- Weighted value
- Expected closing date
- Actual closing date
- Deal owner
- Assigned users
- Lead source
- Status:
 - Open
 - Won
 - Lost
 - Archived
 - Lost reason
 - Won notes
- Priority
- Tags
- Description
- Deal-level custom field answers
- Created by
- Updated by
- Last activity date

The Deal reference should be human-readable if the platform already uses reference numbers.

Example:

- DEAL-000123
- SPONSOR-2026-0012

Use existing numbering conventions where possible.

Deal Creation Flow

1. Select pipeline.
2. Load pipeline stages.
3. Load deal-level custom fields from the pipeline's configuration.
4. Set the default stage automatically.
5. Display standard fields.
6. Display conditional fields dynamically.
7. Add one or more Deal Contacts.
8. Add initial notes if needed.
9. Save the deal.

When a pipeline is preselected from a pipeline detail screen, skip the pipeline selection step.

Deal Validation

- Pipeline is required.
- Current stage must belong to the selected pipeline.
- Custom field validation must follow the Pipeline Type configuration.
- Deal amount must be numeric and non-negative.
- Expected close date must be valid.
- Won and Lost stage transitions must apply appropriate validation.
- Lost reason should be required when moving to a Lost stage.
- Actual closing date should be automatically recorded when entering a Won or Lost stage, unless manually overridden by an authorized user.

8. Deal Contacts

Every deal can have one or more contacts.

A Deal Contact should support:

- First name
- Last name
- Full name
- Email
- Phone
- Mobile

- Job title
- Company
- Country
- City
- Address
- Website
- LinkedIn URL
- Contact type or role
- Is primary contact
- Notes
- Existing platform contact reference, if a reusable contacts module exists
- Custom fields, if applicable

Examples of contact roles:

- Decision Maker
- Finance Contact
- Marketing Contact
- Main Contact
- Technical Contact
- Billing Contact
- Sponsor Representative
- Exhibitor Representative

A deal may have multiple contacts, but only one should be marked as primary unless the business rules allow otherwise.

The system should allow:

- Creating a new contact inside the deal
- Selecting an existing contact
- Editing deal-specific contact details
- Removing a contact from the deal without deleting the global contact
- Marking a contact as primary
- Viewing all deals related to a contact, if global contacts exist

Determine whether contacts should be:

1. Global organization or event contacts with a many-to-many deal relationship, or
2. Deal-specific contact records

Prefer a reusable global Contact entity if the platform already has contacts, attendees, exhibitors, sponsors, or CRM contacts that can safely be reused.

Avoid duplicating contact data unnecessarily.

9. Deal Notes

Each deal must have a Notes section.

A note should contain:

- Deal
- Note body
- Author
- Created date and time
- Updated date and time
- Optional note type
- Optional attachment
- Optional pin status
- Optional visibility, such as internal only
- Mentions, if the platform supports them

Users should be able to:

- Add note
- Edit their own note, according to permissions
- Delete note
- Pin or unpin note
- Upload attachment
- View notes in chronological or reverse chronological order

All note actions should appear in the deal activity log.

10. Deal Detail Screen

Suggested layout:

Header

- Deal title
- Deal reference
- Pipeline
- Current stage
- Deal value
- Owner
- Expected close date
- Status
- Edit button
- Move stage action

- Mark won action
- Mark lost action
- More actions menu

Main Sections or Tabs

- Overview
- Contacts
- Notes
- Activity
- Files
- Form Submission Source, if applicable

Overview

Show:

- Standard deal information
- Custom field values
- Pipeline
- Stage
- Amount
- Probability
- Weighted amount
- Owner
- Assigned users
- Dates
- Tags
- Description
- Created by and updated by

Contacts

Display linked contacts and allow contact management.

Notes

Display and manage deal notes.

Activity

Display a complete audit timeline.

Source Information

If the deal originated from an inquiry form submission, show:

- Form name
 - Submission reference
 - Submission date
 - Original submitted values
 - Link back to the submission
-

11. Deal Stage Movement

Deals can be moved between stages.

Support stage movement through:

- Deal edit form
- Deal detail page
- Kanban drag and drop
- Bulk action from the deal listing, if appropriate

When moving a deal:

1. Validate that the target stage belongs to the deal's pipeline.
2. Check stage-specific required fields.
3. If fields are missing, display a modal asking the user to complete them.
4. Record previous stage.
5. Record new stage.
6. Record moved by.
7. Record moved date and time.
8. Update probability based on the stage, unless the deal has an explicitly overridden probability.
9. If moved to Won:
 10. Set status to Won.
 11. Set actual closing date.
 12. Optionally require won notes or final amount.
13. If moved to Lost:
 14. Set status to Lost.
 15. Set actual closing date.
 16. Require lost reason.

17. If moved back to an Open stage:
18. Reopen the deal.
19. Clear or preserve close dates based on documented business rules.
20. Add an activity log entry.

Create a deal stage history record so reporting can show:

- Time spent in each stage
 - Number of stage changes
 - Previous and current stage
 - Complete movement timeline
-

12. Deals Table View

Create a table view with configurable columns.

Default columns:

- Deal reference
- Deal title
- Pipeline
- Stage
- Primary contact
- Company
- Owner
- Deal value
- Probability
- Weighted value
- Expected close date
- Status
- Last activity
- Created date

Filters:

- Pipeline Type
- Pipeline
- Stage
- Owner
- Assigned user
- Status
- Deal amount range
- Expected closing date

- Created date
- Lead source
- Priority
- Tags
- Contact
- Custom field filters

Features:

- Search
- Sort
- Pagination
- Column selection
- Saved filters, if supported
- Bulk assignment
- Bulk stage movement
- Bulk archive
- Export
- Create deal
- Open deal detail

13. Deals Kanban View

Create a Kanban view for Deals.

The Kanban view should:

- Group deals by pipeline stage
- Use the configured stage order
- Use the configured stage colors
- Show stage totals
- Show total deal value per stage
- Show weighted value per stage
- Show number of deals per stage
- Allow horizontal scrolling
- Allow drag and drop between stages
- Support pagination or lazy loading per column
- Preserve filters
- Be responsive

Each Deal card should display:

- Deal title
- Deal reference
- Primary contact
- Company

- Deal amount
- Currency
- Expected closing date
- Owner
- Priority
- Tags
- Last activity date
- Overdue indicator
- Source form indicator, if applicable

Drag and drop requirements:

- Optimistic UI may be used, but rollback on server failure.
- Validate stage requirements before completing the move.
- If additional information is required, open a modal.
- Prevent unauthorized movement.
- Record activity and stage history.
- Handle concurrent updates safely.
- Return a meaningful error when another user has updated the deal.

The Kanban screen should allow selection of:

- Pipeline Type
- Pipeline
- Owner
- Date range
- Status
- Other supported filters

Prefer requiring a specific pipeline before rendering the Kanban if stages differ between pipelines.

14. Sales Dashboard

Create a basic Sales dashboard using available data.

Suggested widgets:

- Total pipelines
- Open deals
- Open deal value
- Weighted pipeline value
- Won deal value
- Lost deal value
- Conversion rate
- Deals expected to close soon
- Overdue deals

- Deals by stage
- Deals by owner
- Deals by Pipeline Type
- Recent activity
- Recent inquiry submissions
- Form submissions awaiting review
- Top-performing pipelines

Use existing chart libraries and dashboard components.

Ensure all metrics respect organization, event, and permission scope.

Part 2: Website Inquiry Forms

15. Inquiry Forms Overview

The Sales module must include a flexible form builder that can create public forms for event websites.

Examples:

- Exhibitor Inquiry Form
- Group Inquiry Form
- Sponsor Inquiry Form
- Partnership Inquiry Form
- Speaker Inquiry Form
- Vendor Inquiry Form
- Media Partnership Form
- General Sales Inquiry Form
- Custom Inquiry Form

The form builder should reuse or extend the existing custom questions and conditional questions functionality.

Each form should be independently configurable and embeddable on external websites.

16. Inquiry Form General Settings

Each form should contain:

- Form name
- Internal code or slug
- Form type or category
- Description

- Status:
- Draft
- Published
- Unpublished
- Archived
- Event
- Organization
- Form heading
- Introductory text
- Submit button label
- Success message
- Error message
- Confirmation email settings
- Internal notification settings
- Start date and time
- End date and time
- Maximum submission limit
- Allow multiple submissions from the same email
- Require login or allow public access
- CAPTCHA or anti-spam setting
- Terms and conditions checkbox
- Privacy policy checkbox
- Redirect URL after submission
- Language
- Theme or display settings
- Associated Pipeline Type
- Associated Pipeline
- Default deal stage
- Auto-create Deal setting
- Submission assignment rules
- Tags
- Created by
- Updated by

17. Form Builder

Allow administrators to create form fields using drag and drop or the platform's existing question builder.

Supported fields should include:

- Section heading
- Description text
- Short text
- Long text
- Email

- Phone
- Number
- Currency
- Date
- Date and time
- Dropdown
- Multi-select
- Radio buttons
- Checkboxes
- Yes or No
- URL
- File upload
- Country
- Industry
- Company
- Contact name
- Hidden field
- Consent checkbox

Each field should support:

- Label
- Internal key
- Description
- Placeholder
- Required
- Default value
- Validation
- Sort order
- Width or layout size
- Options
- Conditional visibility
- Conditional requirement
- Mapping to Deal field
- Mapping to Deal Contact field
- Mapping to Pipeline custom field, if applicable
- Mapping to Deal custom field
- Whether included in notification email
- Whether included in confirmation email
- Whether visible in submission listing

Conditional logic examples:

- Show "Requested Booth Size" when inquiry type is Exhibitor.
- Show "Sponsorship Budget" when inquiry type is Sponsor.
- Show "Number of Participants" when inquiry type is Group.
- Show "Other Industry" when Industry equals Other.

- Require Company Registration Document when requested package value exceeds a configured amount.

The form builder should validate circular or invalid conditional dependencies.

18. Form Design and Appearance

Provide reasonable styling controls without turning this into a complete page builder.

Suggested settings:

- Form layout:
 - Single column
 - Two columns
- Label position
- Button alignment
- Primary color
- Background color
- Border radius
- Form width
- Custom CSS class
- Show or hide form title
- Show or hide description
- Light or dark mode, if supported

Security note:

If custom CSS is supported, sanitize and scope it carefully.

Do not allow arbitrary JavaScript to be entered through form settings.

19. Form Publishing

A form can be saved as Draft and later Published.

Published forms should have:

- Public URL
- Embed code
- Preview page
- Copy embed code button
- Copy public URL button
- QR code, optional

- Publish or unpublish action

Example public URL:

```
https://events.example.com/forms/{form-slug}
```

Or according to the platform's existing event-domain structure:

```
https://{event-subdomain}.example.com/inquiry/{form-slug}
```

Use the existing public event website routing conventions.

20. Embed Code

The system must generate embed code that users can copy and paste into any website.

Preferred implementation options:

Recommended Option: JavaScript Embed

Generate code similar to:

```
<div id="event-sales-form-UNIQUE_ID"></div>
<script
  src="https://platform-domain.com/embed/sales-form.js"
  data-form-id="PUBLIC_FORM_TOKEN"
  data-target="event-sales-form-UNIQUE_ID"
  async>
</script>
```

The JavaScript should inject the form into the selected container.

Alternative:

```
<script src="https://platform-domain.com/embed/forms/PUBLIC_FORM_TOKEN.js"
  async></script>
```

The embed script may render an iframe internally.

Iframe Fallback

Also provide an iframe code option:

```
<iframe
  src="https://platform-domain.com/embed/forms/PUBLIC_FORM_TOKEN"
  width="100%"
  height="700"
  frameborder="0"
  loading="lazy">
</iframe>
```

Recommended technical approach:

- Render the public form inside an iframe for CSS isolation and security.
- Use a lightweight JavaScript loader to insert the iframe.
- Automatically resize iframe height using `postMessage`.
- Validate the message origin.
- Support responsive width.
- Do not expose internal database IDs.
- Use a public UUID or secure token.
- Allow optional domain restrictions.
- Use Content Security Policy and frame ancestor settings appropriately.
- Avoid allowing the embedded website to access protected platform data.

The embed script should:

1. Read the form token.
2. Find the target container.
3. Create the iframe.
4. Set width to 100%.
5. Set initial safe height.
6. Listen for trusted resize messages.
7. Update iframe height.
8. Show a fallback link if scripts are disabled.
9. Avoid rendering the same form twice accidentally.
10. Handle unavailable, unpublished, expired, or deleted forms gracefully.

On the form detail screen, show:

- JavaScript embed code
 - Iframe embed code
 - Public URL
 - Copy buttons
 - Preview button
 - Domain restriction settings
 - Embed instructions
-

21. Public Form Submission

When a visitor submits a form:

1. Validate the form is published.
2. Validate start and end dates.
3. Validate the submission limit.
4. Validate all visible required fields.
5. Ignore hidden conditional fields that are not currently active.
6. Validate uploads.
7. Apply CAPTCHA or anti-spam checks.
8. Rate-limit submissions.
9. Store the submission.
10. Store submitted field values as structured answers.
11. Store safe request metadata:
12. Submitted date and time
13. IP address, only if permitted by privacy requirements
14. User agent
15. Referrer URL
16. Source page URL
17. UTM parameters
18. Embed domain
19. Send confirmation email if enabled.
20. Send internal notifications if enabled.
21. Optionally create a Deal.
22. Display success message or redirect.

Do not trust field labels or client-side validation. Validate everything server-side using the form's stored field configuration.

Use secure file upload handling.

22. Form Submissions

Every form detail screen must include a Submissions tab or section.

Display:

- Submission reference
- Submission date
- Submitter name

- Email
- Phone
- Company
- Status
- Assigned user
- Deal conversion status
- Associated deal
- Source URL
- Embed domain
- Last updated date

Submission statuses may include:

- New
- In Review
- Contacted
- Qualified
- Converted to Deal
- Rejected
- Spam
- Archived

Filters:

- Status
- Submitted date range
- Assigned user
- Converted or not converted
- Email
- Company
- Source URL
- Embed domain
- Form
- Pipeline
- Custom submitted field values

Features:

- Search
- Sort
- Pagination
- Export
- Bulk status update
- Bulk assignment
- Mark as spam
- Archive
- Delete according to permissions
- Convert to Deal

- Open submission detail
-

23. Submission Detail Screen

Display:

- Form name
- Submission reference
- Submission status
- Submitted date and time
- Submitter details
- All submitted answers
- Uploaded files
- Source URL
- Embed domain
- UTM values
- IP and user agent, subject to privacy requirements
- Assigned user
- Internal notes
- Linked deal
- Conversion history
- Activity history

Actions:

- Change status
 - Assign user
 - Add internal note
 - Send email, only if an email module exists
 - Convert to Deal
 - Link to existing Deal
 - Mark as spam
 - Archive
 - Delete
-

24. Convert Submission to Deal

A form may be associated with:

- A Pipeline Type
- A specific Pipeline
- A default deal stage

The system should support both manual and automatic conversion.

Manual Conversion

From the submission detail screen:

1. Click "Convert to Deal".
2. Select a pipeline, unless already configured.
3. Load the appropriate deal fields.
4. Prepopulate mapped fields from form answers.
5. Prepopulate contact details.
6. Select the deal owner.
7. Select the initial stage.
8. Review and edit values.
9. Create the deal.
10. Link the submission to the deal.
11. Mark the submission as Converted to Deal.
12. Record the activity.

Automatic Conversion

When enabled in form settings:

1. Create a Deal after successful form submission.
2. Use the selected pipeline.
3. Use the configured default stage.
4. Map form fields to standard and custom deal fields.
5. Create or link the primary contact.
6. Assign the deal using the configured assignment rule.
7. Link the Deal to the original submission.
8. Send internal notification if configured.

Prevent duplicate conversion.

A submission should not create multiple deals unless explicitly allowed.

25. Field Mapping

The form builder should allow mapping form fields to:

Standard Deal Fields

- Deal title
- Deal value

- Expected close date
- Description
- Lead source
- Priority
- Tags

Standard Contact Fields

- First name
- Last name
- Email
- Phone
- Job title
- Company
- Website
- Country
- City

Deal Custom Fields

Based on the associated Pipeline Type.

The mapping UI should:

- Show only compatible field types.
- Prevent two fields from incorrectly overwriting the same target unless explicitly allowed.
- Warn about required Deal fields that have no source mapping.
- Allow fixed default values.
- Allow simple value transformations where needed.
- Store mapping configuration in a structured format.

Example:

- Form field `company_name` → Contact company
- Form field `contact_email` → Primary Contact email
- Form field `requested_package` → Deal custom field `sponsorship_package`
- Form field `budget` → Deal value
- Form field `message` → Deal description

26. Notifications

Support configurable notifications for:

- New form submission
- Deal created

- Deal assigned
- Deal moved to another stage
- Deal won
- Deal lost
- Deal expected closing date approaching
- Deal overdue
- New deal note or mention

Notification channels should follow existing platform capabilities:

- In-app
- Email
- Other existing channels

Form internal notifications should allow:

- Selected users
- Selected roles
- Pipeline owner
- Specific email addresses, if permitted
- Conditional recipient based on an answer, if feasible

27. Activity Log and Audit Trail

Create a proper audit trail for sensitive Sales actions.

Track:

- Pipeline Type created, updated, archived, or deleted
- Stage created, updated, reordered, or removed
- Pipeline created or updated
- Deal created or updated
- Deal moved between stages
- Deal owner changed
- Deal value changed
- Deal won or lost
- Contact added or removed
- Note added, updated, or deleted
- Inquiry form created, updated, published, or unpublished
- Form submitted
- Submission status changed
- Submission assigned
- Submission converted to Deal
- Deal linked to submission

Store:

- Entity
- Entity ID
- Action
- Old values
- New values
- User
- Timestamp
- Relevant metadata

Reuse the existing activity or audit log implementation if present.

28. Suggested Data Model

Inspect the existing schema first and adjust naming to match conventions.

Potential tables:

- sales_pipeline_types
- sales_pipeline_type_stages
- sales_pipeline_type_fields
- sales_pipeline_type_field_options
- sales_pipeline_type_field_conditions
- sales_pipelines
- sales_pipeline_field_values
- sales_pipeline_stages
- sales_deals
- sales_deal_field_values
- sales_deal_contacts
- contacts, if a global contacts table is introduced or reused
- sales_deal_notes
- sales_deal_stage_history
- sales_deal_activities
- sales_forms
- sales_form_fields
- sales_form_field_options
- sales_form_field_conditions
- sales_form_field_mappings
- sales_form_submissions
- sales_form_submission_answers
- sales_form_submission_files
- sales_form_submission_notes
- sales_form_submission_activities

Every relevant table must include tenancy and scope columns based on current conventions, such as:

- organization_id
- event_id
- created_by
- updated_by
- deleted_at
- timestamps

Use UUIDs or public tokens for externally exposed resources.

Do not expose sequential internal database IDs in public form URLs or embed codes.

Use foreign keys and indexes appropriately.

Potential indexes:

- organization_id
- event_id
- pipeline_type_id
- pipeline_id
- stage_id
- owner_id
- status
- expected_close_date
- form_id
- submission status
- public token
- created_at
- email
- conversion deal ID

Review query patterns before finalizing indexes.

29. API and Backend Design

Follow existing route and controller patterns.

Separate responsibilities clearly.

Potential backend components:

- PipelineTypeController
- PipelineController
- DealController

- DealStageController
- DealContactController
- DealNoteController
- SalesFormController
- SalesFormSubmissionController
- PublicSalesFormController
- FormEmbedController

Use Form Request validation classes.

Use Policies or Gates.

Use services or actions for complex workflows, such as:

- CreatePipelineFromType
- MoveDealToStage
- CreateDeal
- ConvertSubmissionToDeal
- SubmitPublicSalesForm
- PublishSalesForm
- GenerateEmbedConfiguration
- SyncPipelineTypeConfiguration

Use database transactions for:

- Pipeline creation
- Deal creation with contacts
- Stage movement
- Form submission
- Submission conversion
- Field mapping operations

Use events for side effects:

- PipelineCreated
- DealCreated
- DealStageChanged
- DealWon
- DealLost
- FormSubmitted
- SubmissionConvertedToDeal

Use queued listeners for email notifications and other non-blocking operations.

30. Frontend Requirements

Use existing frontend conventions.

The UI should include:

- Responsive layouts
- Loading states
- Empty states
- Validation errors
- Confirmation dialogs
- Toast notifications
- Accessible labels
- Keyboard-friendly controls
- Mobile-friendly tables or cards
- Drag and drop for stage and field ordering
- Drag and drop for Kanban deals
- Unsaved changes warning in builders
- Preview mode for forms

Avoid building duplicate components when reusable components already exist.

Potential reusable components:

- DynamicFieldRenderer
- ConditionalFieldEngine
- FormBuilder
- FieldSettingsPanel
- StageBuilder
- PipelineSelector
- DealCard
- DealKanbanBoard
- ContactSelector
- NotesTimeline
- ActivityTimeline
- EmbedCodePanel
- CopyToClipboard
- SubmissionAnswersViewer

31. Multi-Tenancy and Security

This module must strictly follow existing tenancy rules.

Every backend query must be scoped appropriately.

Never trust:

- organization_id
- event_id
- pipeline_id
- stage_id
- form_id
- deal_id
- submission ID

Validate all relationships within the authenticated tenant and event.

Public form requirements:

- Use secure public form tokens.
- Do not expose administrative fields.
- Rate-limit submissions.
- Validate file uploads.
- Sanitize displayed content.
- Escape user-generated values.
- Protect against XSS.
- Protect against CSRF where applicable.
- Protect against spam.
- Validate embed origins where domain restrictions are enabled.
- Use signed URLs for private files.
- Avoid exposing submitter metadata publicly.
- Ensure an unpublished form cannot accept submissions.
- Add honeypot support if appropriate.
- Add CAPTCHA integration using an existing provider, if available.

32. Performance Requirements

Plan for pipelines with thousands of deals and forms with thousands of submissions.

Use:

- Pagination
- Lazy loading
- Efficient eager loading
- Indexed queries
- Aggregated queries for dashboards
- Debounced search
- Server-side filtering
- Cached form configuration for public forms
- Queued notifications

- Efficient Kanban loading

Avoid loading all deals for every Kanban column at once.

Consider:

- Initial limited records per stage
 - “Load more” per stage
 - Server-side stage totals
 - Server-side stage value totals
-

33. Reporting and Export

At minimum, support exports for:

- Deals
- Deal contacts
- Pipeline summary
- Form submissions

Use the platform’s existing CSV or Excel export system.

Exports should respect:

- Active filters
- Selected columns
- Permissions
- Tenant and event scope
- Custom fields

Potential future reports should be supported by the data model:

- Pipeline conversion rate
- Stage conversion rate
- Average time in stage
- Sales cycle duration
- Won revenue by owner
- Lost reasons
- Deal source performance
- Form conversion rate
- Form-to-deal conversion
- Revenue by Pipeline Type

Do not necessarily implement every advanced report in the first version, but make sure the schema supports them.

34. Testing Requirements

Add automated tests following existing project standards.

At minimum, test:

Pipeline Types

- Authorized user can create Pipeline Type.
- Unauthorized user cannot create Pipeline Type.
- Stages are created and ordered correctly.
- Only one default stage exists.
- Used stages cannot be deleted unsafely.
- Custom field configuration is saved correctly.

Pipelines

- Pipeline can be created from Pipeline Type.
- Pipeline stages are copied correctly.
- Pipeline-level fields are validated.
- Pipeline is scoped to the correct organization and event.

Deals

- Deal can be created.
- Deal fields are loaded from the correct Pipeline Type.
- Deal cannot use a stage from another pipeline.
- Deal stage can be changed.
- Required fields are enforced during stage movement.
- Lost reason is required for a Lost stage.
- Stage history is created.
- Contacts can be added.
- Notes can be added.
- Unauthorized users cannot view or update the deal.

Forms

- Draft form cannot be submitted publicly.
- Published form can be submitted.
- Expired form cannot be submitted.
- Conditional required fields are validated correctly.
- Invalid file uploads are rejected.
- Submission metadata is stored correctly.
- Rate limits work.
- Tenant leakage is prevented.

Conversion

- Submission can be converted to a Deal.
- Field mapping works.
- Contact mapping works.
- Duplicate conversion is prevented.
- Automatic conversion works.
- Submission and Deal remain linked.

Embed

- Valid token loads the form.
- Invalid token returns a safe error.
- Unpublished form is unavailable.
- Domain restrictions work.
- Resize message validates origin.

Use factories and seeders for test data.

35. Seeder and Demo Data

Create optional development seeders for:

Pipeline Types

Sponsorship Sales

Stages:

- New Inquiry
- Contacted
- Qualified
- Proposal Sent
- Negotiation
- Won
- Lost

Deal fields:

- Company
- Sponsorship Package
- Budget
- Industry
- Expected Close Date
- Requirements

- Lost Reason

Exhibitor Sales

Stages:

- Inquiry Received
- Qualification
- Booth Options Shared
- Quotation Sent
- Follow-up
- Confirmed
- Lost

Deal fields:

- Company
- Booth Size
- Hall Preference
- Product Category
- Budget
- Country
- Expected Close Date

Forms

- Sponsor Inquiry Form
- Exhibitor Inquiry Form
- Group Inquiry Form

This demo data must only run in development or when explicitly requested.

36. Development Approach

Do not immediately begin coding the entire feature.

Follow this sequence:

Step 1: Codebase Analysis

Inspect:

- Existing architecture
- Tenancy
- permissions

- Event scoping
- Custom question engine
- Conditional question engine
- Public form handling
- Existing contact entities
- Existing audit logs
- Existing notifications
- Existing export implementation
- Existing frontend components
- Existing test patterns

Summarize what can be reused.

Step 2: Implementation Plan

Create a detailed implementation plan divided into phases.

Recommended phases:

Phase 1: Foundation

- Permissions
- Database design
- Models
- Enums
- Relationships
- Policies
- Basic routes

Phase 2: Pipeline Types

- Pipeline Type CRUD
- Stage builder
- Pipeline-level fields
- Deal-level fields
- Conditional logic

Phase 3: Pipelines

- Pipeline CRUD
- Pipeline creation from type
- Pipeline-specific stages
- Pipeline detail

Phase 4: Deals

- Deal CRUD
- Contacts

- Notes
- Stage movement
- Stage history
- Activity log
- Table view

Phase 5: Kanban

- Kanban board
- Drag and drop
- Stage validation
- Lazy loading
- Aggregates

Phase 6: Inquiry Forms

- Form CRUD
- Form builder
- Conditional fields
- Publishing
- Public URLs

Phase 7: Embed System

- Embed token
- JavaScript loader
- Iframe renderer
- Auto-resize
- Domain restrictions

Phase 8: Submissions

- Public submission processing
- Submission table
- Submission detail
- Notifications
- Exports

Phase 9: Deal Conversion

- Field mapping
- Manual conversion
- Automatic conversion
- Contact mapping
- Source linkage

Phase 10: Dashboard and Reporting

- Dashboard metrics

- Sales widgets
- Submission analytics

Phase 11: Tests and Hardening

- Feature tests
- Unit tests
- Authorization tests
- Tenancy tests
- Performance review
- Security review

Step 3: Identify Risks and Decisions

Before coding, explicitly document decisions regarding:

- Copying versus referencing Pipeline Type stages
- Global versus deal-specific contacts
- Reusing versus extending existing custom questions
- Public token format
- Embed implementation
- Form configuration storage
- Custom field answer storage
- Conditional rule storage
- Deal activity storage
- Audit log integration
- Form submission retention
- File storage
- Auto-conversion behavior
- Duplicate contact matching
- Duplicate submission detection

Step 4: Implement Incrementally

Implement one phase at a time.

After every phase:

- Run migrations
- Run tests
- Run static analysis or linting
- Verify permissions
- Verify tenant scoping
- Summarize changed files
- Summarize remaining work
- Identify any blockers

Do not leave incomplete placeholder code without clearly marking it.

37. Acceptance Criteria

The feature is considered complete when:

1. Administrators can create Pipeline Types.
 2. Administrators can define ordered stages.
 3. Administrators can define pipeline-level questions.
 4. Administrators can define deal-level custom fields.
 5. Conditional fields work.
 6. A pipeline can be created from a Pipeline Type.
 7. Pipeline-specific questions appear during pipeline creation.
 8. Pipeline stages are available after pipeline creation.
 9. Users can create deals under a pipeline.
 10. Deal fields match the selected pipeline's configuration.
 11. Deals can have multiple contacts.
 12. Deals can have notes.
 13. Deals can be moved between stages.
 14. Stage-specific validation works.
 15. Stage history is recorded.
 16. Deals can be viewed in a table.
 17. Deals can be viewed in a Kanban board.
 18. Deals can be dragged between stages.
 19. Users can create public inquiry forms.
 20. Forms support custom and conditional questions.
 21. Forms can be published and unpublished.
 22. Forms have public URLs.
 23. Forms provide JavaScript and iframe embed codes.
 24. Embedded forms work on external websites.
 25. Public submissions are validated and stored.
 26. Form submissions appear on the form detail screen.
 27. Submission details can be reviewed.
 28. Submissions can be manually converted into Deals.
 29. Forms can automatically create Deals.
 30. Form answers can map to Deal and Contact fields.
 31. All operations respect permissions.
 32. All operations respect tenant and event scope.
 33. Important activities are logged.
 34. Automated tests cover the critical flows.
 35. The implementation follows the existing platform architecture.
-

38. First Response Required from Cursor

Before making any code changes, provide:

1. A summary of the existing relevant architecture.
2. Components and modules that can be reused.
3. Proposed database schema.
4. Proposed models and relationships.
5. Proposed backend classes.
6. Proposed routes and permissions.
7. Proposed frontend pages and components.
8. Proposed development phases.
9. Technical risks and architectural decisions.
10. Any assumptions being made.
11. The first implementation phase and exact files expected to change.

After presenting the plan, begin with the foundation phase unless a critical architectural blocker is discovered.

Do not ask broad or unnecessary questions. Use the codebase to resolve implementation details wherever possible. Where requirements are genuinely ambiguous, make the safest scalable assumption, document it, and proceed.